架构设计：从质量属性到架构决策

从架构图到可审查、可验证、可演进的结构决策

架构设计不是选一个流行技术，而是在约束下做出可解释、可审查、可演进的结构决策。

核心问题

架构决策在哪里影响最大?

本讲产出

架构方案 V1 + ADR 至少 3 条

课程连接

从 C4 模型表达进入可防御的架构决策

课程链条

从结构表达到架构决策

第5讲让系统结构可见，本讲要让关键选择有理由、有取舍、有记录。

上一讲
C1/C2/C3、关键场景时序图

→

本讲
质量属性权衡、架构风格选择、ADR

→

下一讲
以架构方案为基准验证质量属性

承接
C2 图说明系统如何组织，ADR 说明为什么这样组织。

本讲动作
课堂提问：上一讲画图后，哪些选择仍然没有说清楚？

M3 期中评审：看图，更看决策理由

M3 的重点不是图漂亮，而是质量属性、约束条件和架构选择之间能否闭环。

模型包 V1

C1/C2 图更新版 + 关键场景时序图

架构方案 V1

整体结构、质量属性权衡、一致性边界

ADR 至少 3 条

每条必须包含放弃方案、风险和观察指标

M3 输出不是多交几张图，而是交出可被追问的架构理由。

评分锚点：ADR 的“放弃方案”栏不能空。

两位工程师的分水岭

小任务看编码能力，大系统改动看架构判断力。

A类任务

1000行脚本，两天完成，功能正确

B类任务

20万行系统，8人维护，跨模块改动，两周上线

分化原因

不是会不会写代码，而是能否判断边界、风险和验证方式

教师讲解抓手

同样会写代码的人，在大系统里表现差距来自结构理解。

复杂性来自四类力量叠加

软件架构要控制的不是单一复杂度，而是需求、协作、技术和运行约束的叠加。

需求规模与变化

对象多、规则多、优先级变化快

团队协作规模

规则必须显式化，边界必须被约定

技术组合深度

框架、DB、缓存、MQ、认证、AI服务叠加

运行交付约束

高峰流量、故障恢复、日志审计、发布不中断

定义

架构是什么：关键结构决策的集合

架构关心的是系统主要组成单元、协作方式、依赖组织和质量属性保障。

关键

影响范围大、回退代价高、长期影响系统演进

结构

模块划分、层次关系、数据流向、接口约定

决策

不仅说明选了什么，还说明为什么、放弃什么

工程例子

是否统一权限模型、是否异步通知、是否拆服务，都是架构决策。

课堂锚点

名词解释：软件架构。

定义

架构不是技术栈清单

React、Spring Boot、MySQL 是实现手段，不是架构理由。

技术栈清单

列出用了什么框架、数据库和中间件

→

架构决策

说明为什么这样分层、哪些链路强一致、为什么通知异步

简答题：为什么技术栈清单不能代替架构方案？

质量属性才是架构决策的驱动力

功能说明系统做什么，质量属性决定系统必须怎样被组织。

性能

缓存、异步、读写分离

可用性

冗余、熔断、降级

可维护性/可测试性

模块边界、依赖注入、接口清晰

安全性/可演进性

权限模型、扩展点、依赖治理

质量属性之间存在张力

架构权衡不是让所有目标同时最大化，而是明确哪一项优先、为此牺牲什么。

性能 vs 一致性

缓存和异步提升性能，但增加一致性治理

可用性 vs 一致性

故障时保可用可能接受短暂不一致

安全性 vs 性能

认证、授权、审计会增加处理链路

可维护性 vs 短期速度

清晰边界需要前期设计成本

工程产物

架构工作的六类工程产物

架构必须落到可沟通、可评审、可追踪的材料中。

C1上下文图

边界、用户、外部系统

C2/C3结构图

模块、容器、职责和依赖

关键时序图

复杂场景的协作过程

部署视图

运行位置、进程、网络和故障点

过程

架构设计五步法

架构不是凭感觉拍板，而是从驱动因素到评审迭代的过程。

1 识别驱动
功能、质量属性、约束

→

2 关注点
找出最难、影响最大的设计问题

→

3 选择风格
匹配问题域和约束

→

4 描述评审
用视图和 ADR 让选择可质疑

→

5 验证迭代
PoC、评审、指标观察

案例

先识别 5人/6周/机房/无运维，再讨论是否微服务。

考试锚点

简答题：架构设计为什么要先识别驱动因素？

ADR：记录决策，而不是记录结果

ADR 的价值在于保留当时的背景、取舍、风险和观察指标。

背景

这个决策面对什么场景和约束

决策

最终选择了什么

放弃方案

考虑过什么但决定不用，为什么

风险与指标

已知风险是什么，后续看什么数据

ADR-001 示例：首版采用模块化单体

一份合格 ADR 必须展示选择、放弃方案、风险和观察指标。

Architecture Decision Record

背景	5人团队、6周首版、业务闭环尚未验证
决策	采用模块化单体，内部按业务模块划分边界
放弃方案	微服务复杂度过高；纯单体无演进空间
观察指标	P95响应时间、部署耗时、新成员接手时间

ADR

为什么“放弃方案”最重要

只写“我选了 X”无法证明权衡；写清“为什么不用 Y”才体现架构判断。

空白风险

可能只是没想过其他方案

权衡证据

说明比较了可行方案和约束

可审查性

别人可以质疑理由是否成立

可追溯性

未来条件变化时知道何时重评

ATAM：用质量场景审查架构

架构评估不是问“好不好”，而是用具体质量场景找敏感点和权衡点。

呈现架构
说明结构和关键决策

→

收集质量场景
性能、可用性、安全、演进等

→

分析架构方法
逐个场景看支撑能力

→

识别敏感/权衡点
找出影响最大的决策

→

输出风险清单
形成后续验证和改进项

案例

高峰期 500 并发预约响应 <2s，可用于审查缓存、事务和部署决策。

考试锚点

简答题：ATAM 输出什么？

架构风格按问题域理解

不要按名词背架构风格，要先问它解决哪类问题。

结构组织类

分层、前后端分离、模块化单体、六边形

扩展复用类

组件化、插件化、DDD 限界上下文

分布运行类

SOA、微服务、事件驱动、CQRS、云原生

工程例子

分层和微服务不是同一维度；一个系统可同时分层并模块化。

课堂锚点

考试锚点：给定风格，说明解决的问题域。

分层架构：让变化不要互相污染

分层的价值不在层数，而在职责边界和依赖方向。

表示层

UI/API，不写核心业务规则

应用层

用例协调、事务边界、流程编排

领域层

业务规则和状态变化

基础设施层

数据库、消息、外部接口

模块化单体：首版不是退而求其次

单体不等于落后，问题是内部边界是否清晰、依赖是否受控。

适用

团队小、工期短、业务边界尚在验证

做法

按业务模块划分包边界，通过接口协作

优势

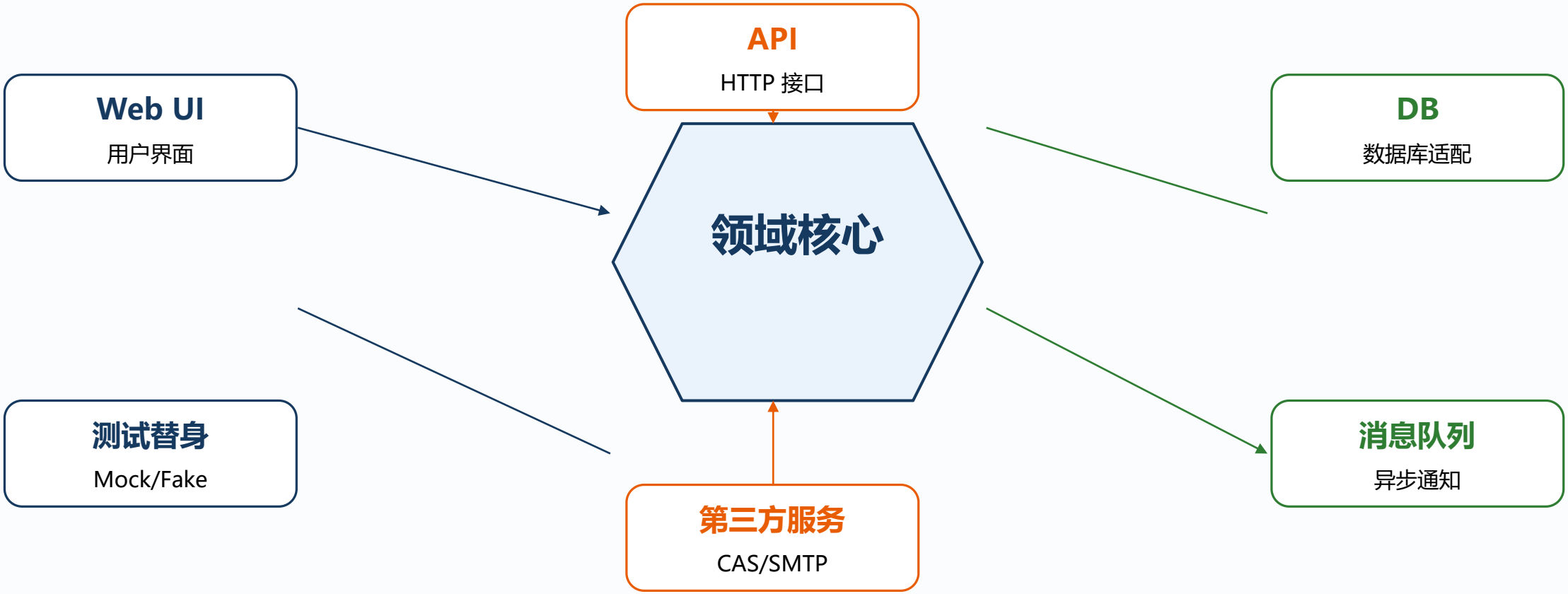
部署简单、事务直接、调试路径短

风险

边界不守会退化成大泥球

六边形架构：端口与适配器

把业务核心与数据库、Web框架、消息队列、外部API隔离，提升可测试性和可演进性。



名词解释：六边形架构。

扩展复用类：组件化、插件化与 DDD

当变化集中在某些能力或业务边界上，要把稳定核心和可变部分分开。

组件化

沉淀可复用能力，如权限、通知、审计

插件化

让扩展能力以插件接入，不改核心

DDD战略设计

用限界上下文划分业务语言和职责

上下文映射

明确不同业务边界之间如何协作

分布运行类：服务化不是默认答案

SOA、微服务、事件驱动、CQRS、云原生解决的是规模化运行和组织协作问题。

SOA/微服务

独立部署、独立数据、独立发布

事件驱动

通过事件解耦主链路和副作用

CQRS

读写分离以支持复杂查询或扩展

云原生

容器、弹性、自动化交付和治理

反模式：分布式单体

拆成多个服务不等于微服务；不能独立部署、独立数据、独立发布，就是分布式单体。

现象

8个服务共用数据库，大量同步调用

后果

发布必须一起发，故障必须整体回滚

根因

没有识别限界上下文，只是按代码目录拆服务

启发

边界模糊时先做好模块化单体

演练准备

约束卡片：本项目到底处在什么条件下

架构选择必须援引约束，不能只说某种技术更先进。

团队规模

5 人

首版工期

6 周

部署环境

学校机房，无公有云

运维资源

无专职运维，由开发兼任

业务阶段

首版，模块边界尚未完全验证

演练规则：每条论据必须引用至少一条约束。

冲突问题：模块化单体还是微服务

这不是先进与落后的比较，而是在约束集合下哪个决策更可防御。

模块化单体

部署简单、事务直接、边界可渐进调整

→

微服务

独立部署、独立扩展、团队自治能力更强

课堂辩论：支持哪一边，必须给约束证据。

课堂演练：架构方案辩论

演练目标不是赢辩论，而是训练用约束和质量属性支撑架构决策。

正方
支持模块化单体

反方
支持微服务

规则
每条论据必须援引约束卡片

课堂动作：分组准备论据，写入 ADR 草稿。

课堂演练

讲评论据：什么叫可防御的架构选择

可防御的论据必须同时连接约束、质量属性和放弃方案。

好论据	5人/6周/无运维 → 首版模块化单体更可控
弱论据	微服务先进、扩展性好，但没有说明当前扩展压力
追问	哪条约束支持首版必须独立部署？
落点	把辩论结论写进 ADR 的背景和放弃方案

评分锚点：论据是否援引约束。

课堂演练

本案例首版结论：模块化单体更可防御

在当前约束下，模块化单体不是保守，而是主动降低不必要复杂度。

理由一

边界尚未验证，过早拆服务会固化错误边界

理由二

无专职运维，分布式治理成本过高

理由三

强一致预约链路在单体内更容易实现和测试

演进

保留通知、统计等后续服务拆分路径

分布式系统：复杂度不是免费午餐

分布式带来独立扩展与发布，也带来网络、数据、测试和运维复杂度。

收益

独立部署、独立扩展、故障隔离、团队自治

代价

网络不可靠、数据一致性、链路定位、版本兼容、测试复杂

判断

没有治理能力，服务化会放大复杂性

工程例子

通知服务拆出去后，主链路和通知链路之间必须设计重试与可观测性。

课堂锚点

简答题：微服务带来哪些额外复杂度？

CAP：真实系统通常在 CP 与 AP 间取舍

真实网络中分区容忍 P 无法放弃，因此实际选择是保一致还是保可用。

C 一致性
所有节点看到同一最新数据

A 可用性
每个请求都能得到响应

P 分区容忍
网络分区时系统仍能继续工作

名词解释：CAP；案例题：区分 CP/AP 场景。

预约系统里哪些场景必须强一致

一致性不是开关，而是按业务后果分场景选择。

A 预约时间段占用

强一致，否则双人预约同一设备

B 审批结果通知

最终一致，晚几分钟可接受

C 设备故障状态更新

强一致，故障设备不能继续被预约

D 审计日志写入

最终一致，短暂延迟不影响主链路

一致性谱系：不止强一致与最终一致

一致性有强弱层次，架构设计要让业务后果与一致性强度匹配。

强一致/线性一致

读到最新写入，代价最高

会话一致

同一用户会话内读到自己的写入

最终一致

短时间可能不一致，最终收敛

选择标准

不一致会不会造成不可接受业务后果

分布式系统五大挑战

微服务不是拆完就结束，真正困难在治理这些系统性挑战。

网络不可靠

超时、重试、幂等

数据一致性

事务边界、Saga、补偿

故障定位难

日志、指标、链路追踪

版本兼容

接口演进、灰度发布

治理工具箱：限流、熔断、降级、幂等、可观测

分布式治理不是锦上添花，而是服务化系统的基础设施。

限流
保护系统不被流量打穿

熔断
依赖故障时快速失败

降级
保核心链路，牺牲非核心能力

幂等
重试不产生重复副作用

可观测性
日志、指标、链路追踪定位问题

治理原则

通知服务失败时预约主链路降级，通知进入重试队列。

考试锚点

简答题：熔断和降级分别解决什么？

综合案例

预约系统关键架构决策汇总

一份架构方案要把决策、理由和放弃方案放在同一张表里。

整体结构	模块化单体；放弃微服务
一致性边界	预约+占用强一致，通知异步
权限控制	后端统一验证，放弃前端隐藏按钮

综合题：为项目写关键架构决策表。

技术选型四步法

技术选型是工程决策，不是技术信仰竞争。

1 问题定义
到底要解决什么问题

→

2 多维调研
能力、成本、团队、生态、风险

→

3 PoC验证
用最小原型验证关键假设

→

4 决策与ADR
记录选择、放弃方案、风险和指标

案例

是否引入 Kubernetes，先问当前是否真的需要弹性编排。

考试锚点

简答题：为什么选型必须做 PoC？

失败案例：5人MVP上 Kubernetes 全家桶

技术复杂度超前于业务复杂度，会直接吞掉交付能力。

现象

K8s、Helm、Istio、Prometheus 全套上阵

后果

环境搭建耗时3周，项目延期

根因

问题定义错误，把学习新技术当成架构需要

启发

先匹配团队能力和业务规模，再谈先进性

技术债：短期收益与长期利息

技术债不是绝对错误，但必须知道借了什么债、利息是什么、何时偿还。

常见债务

跳过测试、规则写死、循环依赖、老旧依赖

债务利息

修改变慢、连锁错误、新人难接手、发布紧张

正当债务

原型阶段有意简化，但记录风险和偿还计划

治理

识别、评估、排期、自动化控制

架构结构决定工程效率

清晰边界、受控依赖、可测试性和可观测性直接影响团队日常效率。

模块边界清晰

可以并行开发，减少互相踩踏

依赖方向受控

局部修改不引发全局连锁

高可测试性

自动化测试快且稳定

接口契约明确

前后端与服务间可并行

CI/CD 与架构协同

持续交付不是流水线工具问题，而是架构是否支持快速构建、测试和回滚。

持续集成
每次提交自动构建和测试

→

持续交付
通过测试的产物随时可发布

→

架构前提
测试隔离、配置外置、数据库变更可管理

→

风险
架构耦合重，流水线越慢越脆

工程例子

模块化程度越高，测试隔离越容易，CI 速度越快。

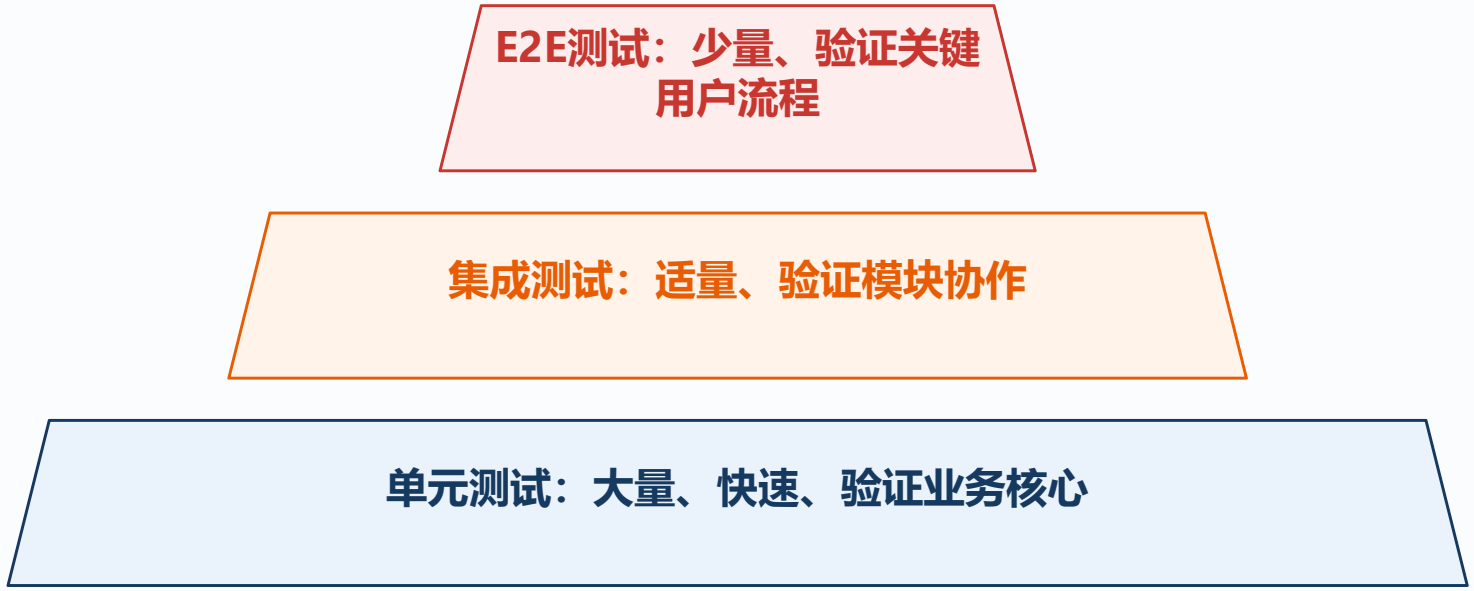
课堂锚点

判断题：买了 Jenkins 就等于有 CI/CD 吗？

工程效率

测试金字塔：架构必须支持低成本验证

如果单元测试很难写，通常是业务逻辑与外部依赖耦合太紧。



与第7讲连接：可测试性由架构决定。

可观测性三支柱

上线后能不能定位问题，取决于日志、指标和链路追踪是否在架构阶段设计。

日志 Logs

结构化 JSON、追踪ID、等级分类

指标 Metrics

QPS、P95/P99、错误率、队列积压

链路追踪 Traces

一次请求跨服务完整路径

工程例子

预约慢了，要能知道慢在网关、预约服务、数据库还是通知链路。

课堂锚点

名词解释：可观测性。

AI 编程让架构判断更重要

AI 加快代码生产，但也会更快放大错误架构带来的技术债。

AI擅长

样板代码、测试骨架、文档、错误解释

AI不擅长

业务驱动识别、组织约束权衡、长期风险判断

新后果

代码生成更快，错误方向积累技术债更快

架构师价值

从写代码转向做高质量判断和约束设计

AI

LLM 集成带来的架构挑战

把 LLM 接入系统，不只是调一个 API，而是引入非确定性、延迟、上下文和成本治理问题。

非确定性输出

同样输入可能有不同回答，需要评估与护栏

秒级延迟

需要流式响应、异步链路、超时降级

上下文窗口

Prompt、历史、检索结果都占 Token

成本治理

缓存、请求合并、模型选择和限额控制

AI

RAG 架构：让模型使用私有知识库

RAG 用检索补充模型上下文，使回答能基于课程资料、制度文档或业务知识库。

用户问题
输入查询

→

向量化
转成可检索表示

→

相似度检索
从向量数据库取相关片段

→

LLM回答
检索结果 + 原始问题生成回答并给出处

工程风险

实验室制度问答可检索设备手册和预约规则后再回答。

考试锚点

名词解释：RAG。

综合案例

预约系统演进路径：由真实痛点驱动

系统不是最终一定走向微服务，而是每一步演进都由真实痛点触发。

阶段1
6周首版：模块化单体完成核心闭环

→

阶段2
稳固：测试覆盖、通知重试、审计历史

→

阶段3
按痛点拆分：通知/统计压力增大再服务化

→

阶段4
规模化治理：追踪、灰度、服务治理

简答题：为什么演进应由痛点驱动？

把本讲原则落到 M3 架构方案

M3 的架构方案要把质量属性、约束、决策、放弃方案连成一条证据链。

质量属性	并发强一致、通知高可用、权限安全、可维护
约束	5人、6周、学校机房、无专职运维
决策	模块化单体、预约强一致、通知异步、后端权限

M3检查：每个决策都要能追溯到质量属性或约束。

M3 提交清单

评审关注质量属性意识、权衡思维和可防御决策记录。

必须提交

- C1 上下文图更新版
- C2 模块/容器划分图更新版
- 架构方案 V1：结构选择、质量属性权衡、一致性边界

建议补强

- ADR 至少 3 条，必须包含放弃方案
- AI 工具使用报告

本讲关键判断

架构设计是为了在复杂性真正到来时，系统仍然可理解、可开发、可运行、可演进。

1. 架构是关键结构决策，不是技术栈清单
2. 质量属性和约束驱动架构选择
3. ADR 的放弃方案体现权衡思维
4. 模块化单体在当前约束下可能是成熟选择

复习锚点：定义、ADR、CAP、模块化单体 vs 微服务、AI边界。